МИРЭА
Российский технологический университет

Файлы и их метаданные

Мухортова Н.Н.

Цель

В этом разделе вы узнаете, как получить доступ к управляющей информации о файле и к информации о состоянии файла. Вы также узнаете, как можно изменить часть этой информации. Управляющая информация и информация о состоянии файла хранится в отдельной структуре данных, называемой инод (inode).

В этом разделе описываются системные вызовы, которые получают и изменяют информацию в inode. Эти системные вызовы не работают с данными, содержащимися файле. Кроме того вы изучите системные вызовы, которые устанавливают максимальный размер файла и изменяют права доступа существующего файла.

Доступность файла - access(2)

Системный вызов access(2) определяет, доступен ли файл для чтения, модификации, или исполнения и просто его существование. В отличие от других системных вызовов, которые проверяют права доступа файла, access(2) использует реальные идентификаторы пользователя и группы вызывающего процесса, чтобы проверить права доступа файла.

Для проверки прав доступа от имени эффективного идентификатора, необходимо использовать аналогичный вызов eaccess(2)

Аргументы access(2)

path Абсолютное или относительное путевое имя файла. Как для всех остальных системных вызовов, каждая директория из путевого имени должна быть доступна на поиск. Иначе системный вызов будет неудачным.

amode Этот аргумент конструируется с помощью побитового ИЛИ из следующих символьных констант, описанных в <unistd.h> :

режим	проверяет
R_OK	чтение
W_OK	изменение
X_OK	исполнение (поиск)
F_OK	существование

Доступность файла - Пример

Эта программа определяет, доступен ли файл для чтения, изменения, исполнения и просто его существование. Она работает следующим образом:

3 Этот препроцессорный макрос определяет количество элементов в массиве table[]. Поскольку препроцессорный макрос — это просто текстовая подстановка, его определение может стоять перед определением самого массива.

8-16 Эта таблица используется для перевода режимов доступа на английский.

18-24 Этот цикл проверяет четыре режима доступа файла.

19-21 Если вызов access(2) успешен для данного режима, печатается соответствующее сообщение.

22-23 Если access(2) неуспешен, то сообщение об ошибке выводится с помощью perror(3).

Доступность файла - Пример

Это пример демонстрируется следующим образом:

$ access access

exists - ok

execute - ok

write: Text file busy read - ok

$ access /etc/passwd

exists - ok

execute: Permission denied

write: Permission denied read - ok

```
1 #include <stdio.h>
2 #include <unistd.h>
3 #define   TABLESIZ   (sizeof(table)/sizeof(*table))
4
5 main(int argc, char *argv[])
6 {
7     int i;
8     static struct {
9      int mode;
10      char *text;
11     } table[ ] = {
12      { F_OK, "exists" },
13      { X_OK, "execute" },
14      { W_OK, "write" },
15      { R_OK, "read" }
16     };
17
18     for (i = 0; i < TABLESIZ; i++) {
19      if (access(argv[1], table[i].mode) != -1)
20          printf("%s - ok\n",
21              table[i].text);
22      else
23          perror(table[i].text);
24     }
25 }
```

Получение и установка ограничений для пользователя

Системный вызов ulimit(2) используется, чтобы определять и устанавливать некоторые ограничения. Этот вызов аналогичен системным вызовам getrlimit(2)/setrlimit(2), но появился гораздо раньше и поддерживается для совместимости со старыми программами. Рекомендуется использовать getrlimit(2)/setrlimit(2).

Встроенная команда shell ulimit(1) также может использоваться для установки этих ограничений

Получение и установка ограничений для пользователя

Аргумент cmd может принять значение одной из следующих символьных констант:

UL_GETFSIZE Возвращает текущее ограничение процесса на максимальный размер файла. Размер измеряется в блоках по 512 байт.

UL_SETFSIZE Устанавливает ограничение на размер файла. Только суперпользователь может увеличить это ограничение. Остальные могут только уменьшить его. Может быть полезно ограничить размер файла при отладке программы, которая создает файлы или удлиняет существующие файлы.

Получение и установка ограничений для пользователя

UL_GMEMLIM Возвращает максимально допустимое значение границы выделяемой памяти. Это значение можно использовать при проверке того, что программа пытается получить память с помощью brk(2) или sbrk(2) больше, чем допустимо. Изменить соответствующий параметр при помощи ulimit(2) невозможно.

UL_GDESLIM Возвращает ограничение, устанавливаемое программно при конфигурации системы, на число файлов, которые процесс может одновременно держать открытыми. newlimit используется при cmd равном UL_SETFSIZE. Это новый размер файла в блоках

Получение и установка маски создания файла

Системный вызов umask(2) используется, чтобы установить параметр cmask (в некоторых документах этот параметр также называется umask): маску ограничения прав доступа к создаваемым файлам или маску создания файла.

Параметр cmask является частью окружения процесса и упоминался в разделе «Файловый ввод-вывод». Shell использует umask(2) при исполнении встроенного оператора umask(1).

Получение и установка маски создания файла

Аргумент cmask - это новое значение маски создания файла. Аргумент, задающий права доступа для open(2), модифицированный с помощью cmask, используется для получения прав доступа файла при его создании.

Права вычисляются по форме mode & (^cmask), где mode — третий параметр open(2). Таким образом, биты, установленные в cmask, не будут присутствовать в правах доступа.

Получение и установка маски создания файла

Например, если вы хотите разрешить всем в вашей группе и всем остальным читать, но не писать в ваши файлы, тогда вам следует установить cmask равным 022 (в языке C числовые константы, начинающиеся с 0, интерпретируются как восьмеричные)

Установка маски создания файла - Пример

Этот пример демонстрирует, как использовать umask(2). Первый аргумент этой программы - новая маска создания. Остальные аргументы — команда, которая будет исполняться как подпроцесс.

12 Значение новой маски (представленное как восьмеричное число) преобразуется в long с помощью библиотечной функции strtol(3).

13 Устанавливается новое значение маски создания. umask(2) возвращает старое значение.

14-17 Печатается новое и старое значение маски создания файла.

19-23 Создается порожденный процесс. Он наследует родительскую маску. Подпроцесс исполняет программу, заданную в arg[2].

Программа демонстрируется следующим образом:

$ umask

0022

$ who > file1

$ setumask 027 sort -o file2 < file1

Old filemode creation mask: 0022

New filemode creation mask: 0027

$ ls -l file?

-rw-r--r-- 1 imr ustg 1258 Apr 3 14:59 file1

-rw-r----- 1 imr ustg 1258 Apr 3 15:00 file2

$ umask

0022

Опция -o функции sort(1) указывает на файл вывода. Значение маски, установленное с помощью umask(2), действует только во время исполнения программы и не меняет cmask родительского процесса shell, что демонстрируется повторным вызовом команды umask(1)

```c
#include <sys/types.h>
#include <stdio.h>
#include <fcntl.h>
#include <stdlib.h>
#include <sys/stat.h>
#include <unistd.h>

main(int argc, char *argv[])
{
    mode_t newmask, oldmask;

    newmask = strtol(argv[1], (char **) NULL, 8);
        oldmask = umask(newmask);
    printf("Old filemode creation mask: %04o\n",
                oldmask);
    printf("New filemode creation mask: %04o\n",
                newmask);

    if (fork() == 0) {
                execvp(argv[2], &argv[2]);
        perror(argv[2]);
        exit(127);
    }
}
```

Определение атрибутов файла

Системный вызов stat(2) используется для получения информации об атрибутах файла. Для просмотра атрибутов не нужно иметь права доступа на чтение, изменение и исполнение для исследуемого файла. Однако, все директории в путевом имени файла должны быть доступны на поиск.

Системный вызов fstat(2) используется для уже открытых файлов. Системный вызов lstat(2) используется для символических связей (понятие символической связи рассматривается в разделе «Управление директориями»). Если файл не является символической связью, lstat(2) ведет себя так же, как stat(2).

Определение атрибутов файла

У stat(2), fstat(2) и lstat(2) следующие аргументы:

path Путевое имя файла или символической связи. Используется для системных вызовов stat(2) и lstat(2).

fildes Дескриптор открытого файла, используется для системного вызова fstat(2).

buf Указатель на структуру stat. Системный вызов stat(2) заполняет структуру stat информацией о файле. Список полей этой структуры приводится на следующей странице

Атрибуты файла

Структура stat используется для получения информации об атрибутах файла. Поля структуры заполняются системными вызовами stat(2), fstat(2) и lstat(2).

У структуры stat следующие поля:

st_dev Это поле идентифицирует файловую систему, которая содержит заданный файл. Это значение можно использовать как аргумент ustat(2), чтобы получить больше информации о файловой системе.

st_ino Inode-номер заданного файла. Файл однозначно определяется с помощью st_dev и st_ino.

st_mode Это поле содержит биты прав доступа к файлу, тип файла и специальные биты. Это поле обсуждается более подробно на следующих страницах раздела. Биты доступа и специальные биты могут быть изменены с помощью системного вызова chmod(2).

st_nlink Число жестких связей заданного файла. Другими словами, это число ссылающихся на заданный файл записей в директориях. Это понятие подробнее рассматривается в разделе «Управление директориями». Поле изменяется с помощью системных вызовов link(2) и ulink(2).

st_uid Пользовательский идентификатор владельца файла. Он изменяется с помощью системных вызовов chown(2) и lchown(2) (для символических связей).

st_gid Идентификатор группы заданного файла. Он также изменяется с помощью системных вызовов chown(2) и lchown(2) (для символических связей).

st_rdev Это поле применяется только для байт- и блок-ориетированных специальных файлов и является идентификатором устройства, на которое этот файл ссылается.

st_size Размер файла в байтах. Он изменяется при записи в файл. Для специальных файлов этот размер всегда равен нулю.

st_atime Время последнего чтения файла. Для директории это время не изменяется при ее поиске (с помощью cd), но изменяется при просмотре содержания директории (с помощью ls), так как при этом читается файл директории. Это время изменяется следующими системными вызовами: creat, mknod, utime и read.

st_mtime Время последней записи в файл. Это время изменяется следующими системными вызовами: creat, mknod, utime и write.

st_ctime Время изменения состояния файла. Это время не изменяется при чтении и модификации содержимого файла. Это время изменяется следующими системными вызовами: chmod, chown, creat, link, mknod, pipe, unlink, utime и write

Атрибуты файла - st_mode

Поле режима файла, st_mode, разделено на три набора битов: тип файла, специальные биты (setiud/setgid и «липкий» бит) и биты прав доступа. Для интерпретации st_mode используются символьные константы, определенные в

S_IFMT используется для выделения типа файла из поля режима файла. Примеры на следующей странице показывают, как использовать структуру stat и перечисленные символьные константы.

```
<sys/stat.h>

#define S_IFMT 0xF000 /* type of file */

#define S_IAMB 0x1FF /* access mode bits */

#define S_IFIFO 0x1000 /* fifo */

#define S_IFCHR 0x2000 /* character special */

#define S_IFDIR 0x4000 /* directory */ /* XENIX definitions are not relevant to Solaris */

#define S_IFNAM 0x5000 /* XENIX special named file */

#define S_INSEM 0x1 /* XENIX semaphore subtype of IFNAM */

#define S_INSHD 0x2 /* XENIX shared data subtype of IFNAM */

#define S_IFBLK 0x6000 /* block special */

#define S_IFREG 0x8000 /* regular */

#define S_IFLNK 0xA000 /* symbolic link */

#define S_IFSOCK 0xC000 /* socket */
```

Печать состояния файла - Пример

Эта программа печатает информацию о состоянии файла. Распечатываются все поля структуры stat, полученной с использованием stat(2). Программа работает следующим образом:

13 Объявление структуры stat.

15-18 Первый аргумент - имя файла. Второй аргумент — адрес структуры stat. Система заполняет структуру информацией о файле.

19-20 Печатаются имя файла и inode-номер.

Печать состояния файла - Пример

21 Функция prntmode() печатает информацию из моды файла. Текст этой функции приведен дальше в этом разделе.

22-23 Печатаются число жестких связей и размер файла.

24-25 Эти функции печатают идентификатор пользователя и различные временные отметки файла. Тексты этих функций приведены дальше в этом разделе.

```
$ stat stat # executable file
file name: stat        i-number: 6028
regular file    permissions: 755
links: 1       file size: 24899
user ID: 49026 name: jrs       group ID: 46014 group: ustg
last access:          Tue Dec 17 11:49:42 1989
last modification:     Tue Dec 17 10:19:45 1989
last status change:    Tue Dec 17 10:19:45 1989
$ stat /etc # directory
file name: /etc        i-number: 1258
directory      permissions: 775
links: 6       file size: 3024
user ID: 0 name: root   group ID: 3 group: sys
last access:          Tue Dec 17 02:30:16 1989
last modification:     Tue Dec 17 10:25:33 1989
last status change:    Tue Dec 17 10:25:33 1989
```

```
1 #include <stdio.h>
2 #include <sys/types.h>
3 #include <stdlib.h>
4 #include <sys/stat.h>
5 static void prntuser(struct stat *);
6 static void prntimes(struct stat *);
7 static void prntmode(struct stat *);
8
9 /* print the status of a file */
10
11 main(int argc, char *argv[])
12 {
13     struct stat stbuf;
14
15     if (stat(argv[1], &stbuf) == -1) {
16         perror(argv[1]);
17         exit(1);
18     }
19     printf("file name: %s\t\t", argv[1]);
20     printf("i-number: %lu\n", stbuf.st_ino);
21     prntmode(&stbuf);
22     printf("links: %lu\t", stbuf.st_nlink);
23     printf("file size: %ld\n", stbuf.st_size);
24     prntuser(&stbuf);
25     prntimes(&stbuf);
26
27     exit(0);
28 }
```